



Specifica Tecnica

02/04/2026



Revisioni

Ver.	Autore	Modifiche	Data	Verifica
0.7	Besnik Murtezan	Aggiornate descrizioni tecnologie frontend	11/05/2026	Niccolò Feltrin
0.6	Stefano Maso	Modifica sezione LLM	24/04/2026	-
0.5	Giuliano Banchieri	Stesura sezione architettura di deployment e organizzazione container	21/04/2026	Spadiliero Ruben
0.4	Stefano Maso	Aggiunte sottosezioni per l'importazione	10/04/2026	Besnik Murtezan
0.3	Stefano Maso	Stesura sezione 3.1.1.2 e sottosezioni	09/04/2026	Besnik Murtezan
0.2	Stefano Maso	Stesura sezione 3.1, 3.1.1 e sottosezioni relative all'architettura del backend e relativi diagrammi delle classi	08/04/2026	Besnik Murtezan
0.1	Giuliano Banchieri	Prima stesura: struttura del documento, tecnologie, riferimenti, introduzione	02/04/2026	Niccolò Feltrin



Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Riferimenti	6
1.3.1	Riferimenti normativi	6
1.3.2	Riferimenti informativi	6
2	Tecnologie	8
2.1	Tecnologie di codifica	8
2.1.1	TypeScript	8
2.1.2	Vue.js	8
2.1.3	Marked.js	9
2.1.4	CSS	9
2.1.5	Tailwind CSS	10
2.1.6	HTML	10
2.1.7	PHP	10
2.1.8	Laravel	10
2.2	Tecnologie di verifica	10
2.2.1	PHPUnit	10
2.3	Tecnologie per la persistenza dei dati (OPT)	10
2.3.1	PostgreSQL	10
2.4	Tecnologie di supporto	11
2.4.1	Vite	11
2.4.2	Composer	11
2.4.3	nginx	11
2.4.4	PHP-FPM	11
2.4.5	Docker	11
2.4.6	Docker Compose	11
2.5	Modelli LLM esterni	11
3	Architettura	12
3.1	Architettura logica	12
3.1.1	Backend	12
3.1.1.1	Gestione note e Esportazione	13
3.1.1.1.1	NoteController	13
3.1.1.1.2	NoteService	13
3.1.1.1.3	NoteRepositoryInterface	14
3.1.1.1.4	NoteRepository	14
3.1.1.1.5	Note	14
3.1.1.1.6	ImportController	14
3.1.1.1.7	ImportService	15
3.1.1.1.8	ImportStrategyFactory	15
3.1.1.1.9	ImportStrategyInterface	15
3.1.1.1.10	MdImport	15
3.1.1.1.11	ExportController	15
3.1.1.1.12	ExportService	15
3.1.1.1.13	ExportStrategyFactory	16
3.1.1.1.14	ExportStrategyInterface	16
3.1.1.1.15	MdExport	16
3.1.1.1.16	HtmlExport	16
3.1.1.1.17	PdfExport	16
3.1.1.1.18	EditorContentFormatter	16
3.1.1.2	Interazione con LLM ^G	17



3.1.1.2.1	LlmController	17
3.1.1.2.2	LlmService	17
3.1.1.2.3	LlmStrategyFactory	18
3.1.1.2.4	LlmProcessorResponse	18
3.1.1.2.5	LlmStrategyInterface	18
3.1.1.2.6	LlmContext	18
3.1.1.2.7	Strategy concrete	18
3.2	Architettura di deployment	19
3.2.1	Organizzazione dei container	19
3.3	Architettura di dettaglio	19
4	Stato dei requisiti funzionali	20



Indice delle immagini

1	Diagramma delle classi per Gestione note - Esportazione - Importazione	13
2	Diagramma delle classi per Interazione con LLM	17



1 Introduzione

1.1 Scopo del documento

Il presente documento contiene informazioni dettagliate sulle tecnologie utilizzate e sulle scelte progettuali del gruppo Tool-8 per la realizzazione del prodotto **Second Brain** su richiesta dell'azienda **Zucchetti S.p.A.**

Sono fornite descrizioni delle tecnologie scelte per backend, frontend e di supporto, i diagrammi delle classi che rappresentano in modo dettagliato l'architettura logica e una tabella di tracciamento dei requisiti soddisfatti.

1.2 Scopo del prodotto

"Estendere il note-taking con i Large Language Models."

L'obiettivo del capitolato^G C6 **"Second Brain"** è lo sviluppo^G di un applicativo che fornisca all'utente un editor di file MD con funzionalità^G AI che semplifichino la scrittura tramite funzionalità^G come: riassunto, traduzione, critica, correzione, e generazione del testo.

L'utente avrà la possibilità di utilizzare queste funzionalità^G anche tramite interrogazioni in linguaggio naturale che verranno processate da un LLM^G contattato dal sistema.

Il software sarà sviluppato sotto forma di applicazione web dove l'utente potrà creare, salvare, eliminare e modificare le sue note.

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Capitolato^G C6 - Progetto^G Second Brain:
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento progetto^G didattico:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
- Norme di Progetto^G v1.1
- Analisi dei Requisiti^G v1.5

1.3.2 Riferimenti informativi

- Verbali
- Glossario v1.0
- Lezioni del corso di *Ingegneria del software*^G aggiornate al 02/04/2026:
 - *"Progettazione^G software (T6)"*:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
 - *"Progettazione^G e programmazione: Diagrammi delle classi (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2023/Diagrammi%20delle%20Classi.pdf>
 - *"Analisi e descrizione delle funzionalità^G: Diagrammi delle attività (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Diagrammi%20di%20Attivit%C3%A0.pdf>
 - *"Progettazione^G: I pattern architetturali"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Software%20Architecture%20Patterns.pdf>
 - *"Progettazione^G: Il pattern Dependency Injection"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Architetturali%20-%20Dependency%20Injection.pdf>



- "Progettazione^G: Pattern Architeturali per le applicazioni con UI":
<https://www.math.unipd.it/~rcardin/sweb/2022/L02.pdf>
- "Progettazione^G: i pattern creazionali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Creazionali.pdf>
- "Progettazione^G: I pattern strutturali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Strutturali.pdf>
- "Progettazione^G: I pattern di comportamento (GoF)":
https://drive.google.com/file/d/1cpi6r0RMxFtC91nI6_sPrG1Xn-28z8eI/view?usp=sharing
- "Programmazione: SOLID programming":
<https://drive.google.com/file/d/1o1Xun2dVVc3mDiaGyN0FrDJhho031fLQ/view?pli=1>



2 Tecnologie

In questa sezione sono descritte le tecnologie utilizzate per lo sviluppo^G di **Second Brain**. Sono inclusi gli strumenti, i framework^G e le librerie utilizzati per lo sviluppo^G di backend, frontend e della parte di persistenza dei dati ma anche quelli di supporto, di verifica^G e i servizi esterni.

2.1 Tecnologie di codifica

Framework^G e librerie utilizzate per lo sviluppo^G backend e frontend.

2.1.1 TypeScript

TypeScript è un linguaggio basato su JavaScript che introduce la tipizzazione statica, migliorando la manutenibilità e la robustezza del codice. Tra le funzionalità^G offerte permette infatti di definire:

- Tipi Statici
- Interfacce
- Template (Generics)
- Enumerazioni

Il tutto è supportato dal compilatore statico *TSC* (*TypeScript Compiler*) che, inizialmente effettua la validazione^G del codice e dei tipi definiti senza l'esecuzione, e successivamente, in assenza di errori, produce codice JavaScript. Le direttive TypeScript sono disponibili solo in fase di sviluppo^G, poichè la "compilazione" termina comunque con la produzione di codice JavaScript.

Ciononostante, TypeScript ed il relativo compilatore sono uno strumento molto utile per verificare la correttezza del codice in fase di sviluppo^G (grazie anche ad un ricco supporto da parte degli IDE moderni) prima ancora della sua esecuzione, seguendo l'approccio "*fail fast*", ovvero l'obiettivo di trovare errori il prima possibile.

Il corretto utilizzo di TypeScript aiuta quindi a mitigare il debito tecnologico del progetto^G, motivo principale per il quale è stato inserito tra le tecnologie di sviluppo. Altra motivazione è la compatibilità con il framework^G Vue (vedere sezione successiva) utilizzato per lo sviluppo^G dell'interfaccia utente.

- **Versione:** 5.9.3
- **Documentazione:** <https://www.typescriptlang.org/docs/> (Consultato 09/04/2026)

2.1.2 Vue.js

Vue è un framework JavaScript open source per lo sviluppo di interfacce utente ed "SPA" (Single Page Application). Si basa su una logica definita per componenti, in cui ogni elemento dell'interfaccia utente viene implementato come componente singolo e poi integrato all'interno di altri componenti "*contenitori*" fino ad arrivare al componente "*radice*", andando a creare una gerarchia chiara e strutturata dell'intera interfaccia utente. In aggiunta ai componenti vengono definite anche le funzioni "*composables*", richiamabili all'interno dei componenti stessi. Le componenti possono essere sviluppate utilizzando JavaScript come linguaggio di scripting oppure, come effettuato nel nostro progetto^G, sfruttare il supporto a TypeScript per irrobustire l'architettura.

Punto di forza del framework^G è la possibilità di ottenere un'interfaccia utente particolarmente reattiva, grazie alla gestione dinamica dei dati tra i componenti basata sull'utilizzo di **props** ed **emits** (idiomi del framework^G): ogni componente infatti si aggiorna automaticamente in seguito a modifiche avvenute sui componenti collegati ad esso.



Vue inoltre mette a disposizione l'utilizzo del **router**, ulteriore strumento nativo del framework^G, con cui è possibile caricare dinamicamente contenuto all'interno della pagina senza eseguire ulteriori richieste: il documento HTML infatti viene richiesto solamente al primo accesso all'applicazione; una volta ottenuto il documento HTML "scheletro", il contenuto viene dinamicamente caricato e modificato attraverso Vue in base alla navigazione. Visivamente l'utente ha l'impressione di interagire con una "singola" pagina (da cui il termine SPA), in quanto il browser non aggiorna l'intero documento HTML della pagina ma sfrutta le funzionalità^G di Vue per aggiornare solamente le parti necessarie.

La scelta di utilizzare il framework^G Vue per il frontend rispetto a sviluppare unicamente utilizzando JavaScript puro è basata sulle seguenti motivazioni:

- **Organizzazione del codice:** ogni componente è costituito da 3 parti:
 - **template HTML:** contiene i tag HTML necessari al componente, muniti delle direttive di Vue (ex. `v-on` | `v-for` | `v-bind`).
 - **script TypeScript:** contiene la logica di comportamento compresa delle chiamate alle funzioni composables; viene sfruttato il supporto a TypeScript per ottenere componenti architetture più robuste rispetto .
 - **stile CSS:** contiene il foglio di stile per il componente (applicato in maniera globale).

Tale suddivisione permette di avere una gestione più pulita del codice e di localizzare le eventuali modifiche, diminuendo enormemente il rischio di creare l'antipattern "*Big Ball of Mud*" (facilmente raggiungibile in caso si utilizzi solamente JavaScript puro).

- **Riutilizzo di codice:** ogni componente può essere riutilizzato in qualsiasi altro punto dell'interfaccia utente ed eventualmente personalizzato evitando la ripetizione di codice.
- **Manutenibilità del codice:** grazie alla logica "*a componenti*", ogni modifica o estensione di codice è localizzata in un punto unico, velocizzando e semplificando le attività di manutenzione.
- **Risorse disponibili:** la scelta del framework^G è stata basata per buona parte anche su tempo disponibile e capacità dei singoli membri del gruppo. Cofrontato con gli altri 2 principali framework^G per sviluppo^G frontend *Angular* e *React*, **Vue** è stato considerato come il giusto compromesso tra la rigidità di Angular e la flessibilità di React, in quanto offre le funzionalità^G necessarie al progetto^G senza una curva di apprendimento troppo alta che avrebbe impattato le risorse disponibili.
- **Versione:** 3.5+
- **Documentazione:** <https://vuejs.org/guide/introduction.html>

2.1.3 Marked.js

Marked.js è una libreria open-source scritta in JavaScript utilizzata per il parsing e la conversione di testo in formato Markdown. Consente di trasformare contenuti Markdown in HTML in modo rapido, supportando estensioni personalizzate e integrazione lato client e server.

- **Versione:**
- **Documentazione:** <https://marked.js.org/>

2.1.4 CSS

CSS (Cascading Style Sheets) è un linguaggio utilizzato per la formattazione di documenti HTML, XHTML e XML.

- **Versione:** CSS3
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/CSS>



2.1.5 Tailwind CSS

Tailwind CSS è un framework^G CSS open source. Offre una serie di classi CSS "di utilità". Queste classi vengono poi personalizzate e combinate secondo necessità per definire lo stile di ogni elemento.

- **Versione:** 4.2.2
- **Documentazione:** <https://tailwindcss.com/docs/installation/using-vite>

2.1.6 HTML

HTML (HyperText Markup Language) è il linguaggio di markup più utilizzato al mondo per i documenti web. È un linguaggio di pubblico dominio e le sue regole sono dettate da **W3C** (World Wide Web Consortium)

- **Versione:** HTML5
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/HTML>

2.1.7 PHP

PHP è un linguaggio di programmazione lato server progettato per lo sviluppo^G web, utilizzato per implementare la logica lato backend.

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/it/>

2.1.8 Laravel

Laravel è un framework^G open source scritto in PHP utilizzato per lo sviluppo^G di applicazioni web moderne. Adotta il pattern architetturale MVC (Model View Controller) e fornisce strumenti integrati per la gestione del routing, dell'autenticazione e dell'accesso ai dati.

- **Versione:** 12.56.0
- **Documentazione:** <https://laravel.com/docs/13.x>

2.2 Tecnologie di verifica

Strumenti utilizzati per l'analisi del codice prodotto.

2.2.1 PHPUnit

PHPUnit è un framework^G utilizzato per l'analisi del codice PHP tramite test^G di unità. Lo scopo è trovare eventuali errori introdotti con modifiche al codice ed evitare che si verifichi una regressione.

- **Versione:** 11.5.55
- **Documentazione:** <https://phpunit.de/documentation.html>

2.3 Tecnologie per la persistenza dei dati (OPT)

2.3.1 PostgreSQL

PostgreSQL è un sistema di gestione di database relazionali open source compatibile con i sistemi operativi più diffusi.

- **Versione:** 17
- **Documentazione:** <https://www.postgresql.org/docs/>



2.4 Tecnologie di supporto

Strumenti utilizzati per facilitare lo sviluppo^G del prodotto da parte del gruppo.

2.4.1 Vite

Vite è uno strumento di sviluppo^G frontend che funge da dev server e build tool.

- **Versione:** 7.3.1
- **Documentazione:** <https://vite.dev/guide/>

2.4.2 Composer

Composer è un gestore di dipendenze per il linguaggio PHP utilizzato per installare, aggiornare e gestire automaticamente librerie e pacchetti necessari a un progetto.

- **Versione:** 2.9.5
- **Documentazione:** <https://getcomposer.org/doc/>

2.4.3 nginx

nginx è un web server open source leggero e ad alte prestazioni, utilizzabile anche come reverse proxy, load balancer, cache HTTP.

- **Versione:** 1.27
- **Documentazione:** <https://nginx.org/en/docs/>

2.4.4 PHP-FPM

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/en/install.fpm.php>

2.4.5 Docker

Docker^G è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati e leggeri, detti container. Ogni container^G include tutte le dipendenze necessarie al funzionamento del software ed è gestito in modo indipendente rispetto agli altri.

- **Versione:** 29.0.1
- **Documentazione:** <https://docs.docker.com/>

2.4.6 Docker Compose

Docker^G Compose è uno strumento che consente di definire, configurare e gestire applicazioni multi-container. Permette di gestire l'avvio, l'arresto e l'interazione tra più container^G in modo coordinato.

- **Versione:** 2.40.3
- **Documentazione:** <https://docs.docker.com/compose/>

2.5 Modelli LLM esterni

Tra i modelli LLM^G (Large Language Models) disponibili tramite l'endpoint Zucchetti, abbiamo selezionato il modello *Gemma3:4B*, in quanto ritenuto il più adeguato per le funzionalità^G AI richieste, sia in termini di velocità di risposta sia per la qualità e coerenza del contenuto generato.



3 Architettura

In questa sezione è descritta in modo dettagliato l'architettura del nostro prodotto. Verranno illustrate le nostre scelte progettuali e i componenti del nostro sistema.

3.1 Architettura logica

Il progetto^G Second Brain è strutturato in due componenti principali:

- **Backend:** responsabile^G della gestione della *logica di business* e della persistenza dei dati.
- **Frontend:** destinato alla presentazione dei dati ottenuti dal *backend*, il *frontend* consente all'utente di interagire con le funzionalità^G del progetto^G in modo intuitivo e user-friendly.

3.1.1 Backend

Nel *backend* si è scelto di utilizzare il framework^G Laravel, che permette la realizzazione di un servizio API^G REST. L'architettura del backend è suddivisa nei seguenti strati:

- **Presentation Layer:** si occupa del routing delle richieste; i controller fungono da intermediari tra il client e i servizi, gestendo il presentation layer e instradando le richieste del client ai servizi appropriati.
- **Application Layer:** gestisce la *logica di business* dell'applicazione; questo strato è responsabile^G delle operazioni e delle regole di business, garantendo che tutte le operazioni siano eseguite correttamente;
- **Data Access Layer:** strato che gestisce la persistenza dei dati, fornendo i mezzi per salvare, recuperare e manipolare i dati.

La suddivisione in questi strati facilita lo sviluppo^G, la manutenzione e la scalabilità del *backend* del progetto^G Second Brain, conformandosi all'architettura **multi-tier** a 3-tier.

3.1.1.1 Gestione note e Esportazione

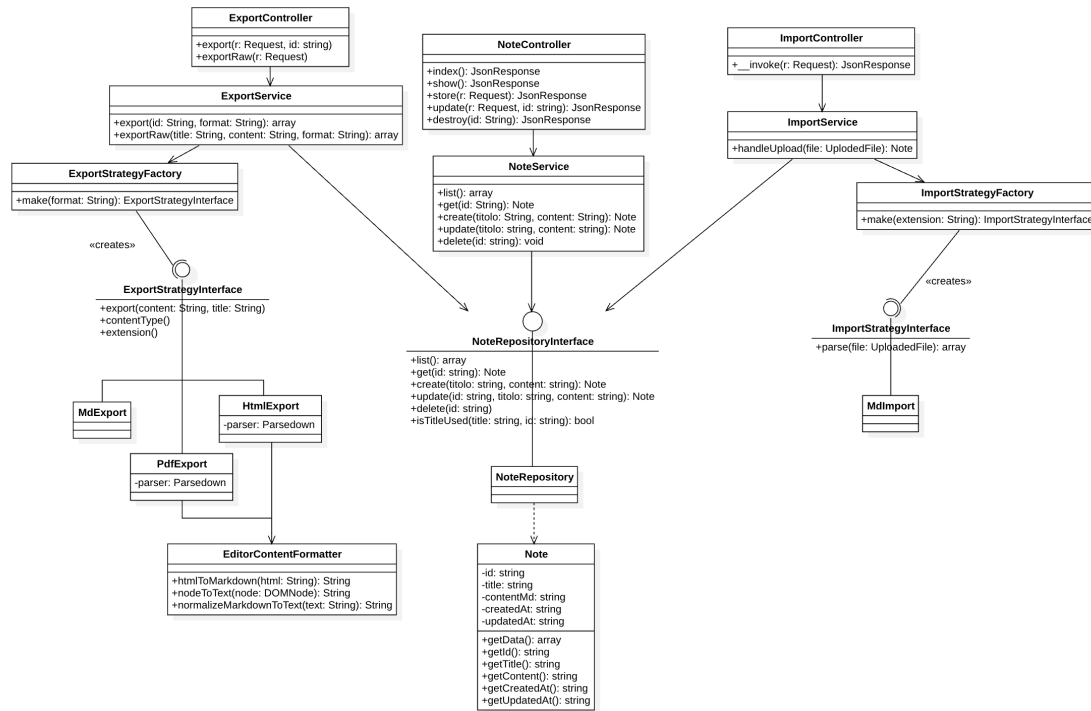


Figura 1: Diagramma delle classi per Gestione note - Esportazione - Importazione

3.1.1.1.1 NoteController È il controller HTTP responsabile^G della gestione delle richieste REST^G relative alle note. Appartiene al layer di presentazione dell'architettura layered e funge da punto di ingresso per tutte le operazioni CRUD espone dall'API. Si limita a validare i dati in ingresso, delegare l'elaborazione a *NoteService* e restituire una risposta HTTP appropriata. Segue la descrizione dei metodi:

- **index()** : **JsonResponse** - gestisce le richieste GET `/notes`, restituendo la lista completa delle note in formato JSON con status 200, delegando il recupero dei dati a *NoteService*
- **show(string \$id)** : **JsonResponse** - gestisce le richieste GET `/notes/{id}` e restituisce i dettagli di una singola nota identificata dall'id fornito
- **store(Request \$request)** : **JsonResponse** - gestisce le richieste POST `/notes`, valida i campi, delega a *NoteService* la creazione e restituisce la nota creata con status 201
- **update(Request \$request, string \$id)** : **JsonResponse** - gestisce le richieste PUT `/notes/{id}`, valida i campi e delega l'aggiornamento a *NoteService*, passando l'id e i nuovi dati
- **destroy(string \$id)** : **JsonResponse** - gestisce le richieste DELETE `/notes/{id}`, delega l'eliminazione a *NoteService* e restituisce una risposta vuota con status 204 in caso di successo

3.1.1.1.2 NoteService È il service responsabile^G della logica di business relativa alle note. Appartiene al layer applicativo dell'architettura layered e funge da intermediario tra il controller e il repository. Dipende esclusivamente dall'interfaccia *NoteRepositoryInterface*, garantendo il rispetto del principio di inversione delle dipendenze. Segue la descrizione dei metodi:

- **list()** : **array** - restituisce la lista completa delle note, delegando direttamente al repository^G
- **get(string \$id)** : **array** - restituisce i dettagli di una singola nota tramite il suo id



- `create(string $title, string $contentMd) : array` - gestisce la creazione di una nuova nota, verificando che il titolo non sia già in uso prima di delegare al repository^G
- `update(string $id, string $title, string $contentMd) : array` - gestisce l'aggiornamento di una nota esistente; prima di delegare al repository^G verifica^G che il titolo non sia già in uso, escludendo dal confronto la nota corrente così che una nota possa essere salvata con il proprio titolo invariato senza incorrere in un falso positivo
- `delete(string $id) : void` - delega l'eliminazione della nota al repository^G

3.1.1.1.3 NoteRepositoryInterface Interfaccia che definisce il contratto che ogni implementazione del repository^G di note deve rispettare. Appartiene al layer di accesso ai dati dell'architettura layered. Dichiarando la dipendenza su questa interfaccia, `NoteService` può funzionare con qualsiasi implementazione senza richiedere modifiche al codice applicativo. Segue la descrizione dei metodi:

- `list() : array` - restituisce la lista completa delle note disponibili, ogni elemento dell'array contiene i metadati della nota senza il contenuto completo
- `get(string $id) : Note` - restituisce i dati completi di una singola nota identificata dall'id fornito
- `create(string $title, string $contentMd) : Note` - crea una nuova nota con il titolo e il contenuto Markdown forniti, genera un identificatore e registra il timestamp di creazione
- `update(string $id, string $title, string $contentMd) : Note` - aggiorna il titolo e il contenuto di una nota esistente assieme al timestamp di ultima modifica
- `delete(string $id) : void` - elimina definitivamente la nota identificata dall'id fornito
- `isTitleUsed(string $title, ?string $excludeId = null) : bool` - verifica^G se un titolo è già utilizzato da un'altra nota; `excludeId` permette di escludere dal confronto una nota specifica, per evitare che tale nota venga bloccata dal confronto con se stessa

3.1.1.1.4 NoteRepository È l'implementazione concreta di `NoteRepositoryInterface` che persiste le note come file Markdown sul filesystem locale tramite Laravel Storage. Realizza tutti i metodi dichiarati nell'interfaccia: `list()`, `get()`, `create()`, `update()`, `delete()`, `isTitleUsed()`; per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.5 Note È una classe Model che rappresenta l'entità Nota nel dominio dell'applicazione. Esistono due tipi di metodo:

- `getData() : array` - serializza l'oggetto in array, utile per passare i dati ai layer superiori
- getter singoli (`getId()`, `getTitle()`, ...) utili quando serve accedere ad una singola proprietà senza serializzare tutto

3.1.1.1.6 ImportController È il controller HTTP responsabile^G della gestione delle richieste di importazione. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il file, delegare l'importazione a `ImportService` e costruire la risposta HTTP. Segue la descrizione dei metodi:

- `__invoke(Request $request) : JsonResponse` - valida il parametro (cioè il campo file della richiesta), delega l'importazione a `ImportService`, passando il file caricato; in caso di successo restituisce la nota importata in formato JSON con status 201, altrimenti ritorna una risposta JSON con status 400.



3.1.1.1.7 ImportService È il service responsabile^G della logica di importazione di una nota. Appartiene al layer applicativo dell'architettura layered. Dipende da `NoteRepositoryInterface` garantendo il disaccoppiamento dall'implementazione del layer di persistenza. Segue la descrizione dei metodi:

- `handleUpload(UploadedFile $file) : Note` - estrae l'estensione del file caricato, delega la creazione della strategy corretta a `ImportStrategyFactory` ed invoca sulla strategy il metodo per estrarre il contenuto in Markdown; successivamente aggiunge un timestamp al titolo della nota per evitare conflitti e invoca `NoteRepositoryInterface::create()`, ritornando un array.

3.1.1.1.8 ImportStrategyFactory - È la factory responsabile^G della creazione della strategy di importazione corretta in base all'estensione del file caricato. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $extension) : ImportStrategyInterface` - metodo statico che istanzia la strategy concreta corrispondente all'estensione del file caricato.

3.1.1.1.9 ImportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di importazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'importazione delle note. Segue la descrizione dei metodi:

- `parse(UploadedFile $file) : array` - estrae il contenuto in Markdown e il titolo della note dal file caricato.

3.1.1.1.10 MdImport È l'implementazione concreta di `ImportStrategyInterface` per l'importazione di file `.md`. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.11 ExportController È il controller HTTP responsabile^G della gestione delle richieste di esportazione, rendendo disponibile una nota come file scaricabile. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il formato richiesto, delega l'esportazione a `ExportService` e costruisce la risposta HTTP con gli header appropriati per il download dei file. Segue la descrizione dei metodi:

- `export(Request $request, string $id)` - valida il parametro (cioè il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando l'id della nota e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404
- `exportRaw(Request $request)` - valida i parametri (cioè il titolo, il contenuto e il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando i diversi parametri e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404

3.1.1.1.12 ExportService È il service responsabile^G della logica di esportazione di una nota in un formato file. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra il repository^G e la factory delle strategy di esportazione. Dipende da `NoteRepositoryInterface`, garantendo il disaccoppiamento dal layer di persistenza. Segue la descrizione dei metodi:

- `export(string $id, string $format) : array` - recupera la nota identificata dall'id tramite il repository^G, delega la creazione della strategy corretta a `ExportStrategyFactory` in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione



- `exportRaw(string $title, string $content, string $format) : array` - delega la creazione della strategy corretta a *ExportStrategyFactory* in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione

3.1.1.1.13 ExportStrategyFactory È la factory responsabile^G della creazione della strategy di esportazione corretta in base al formato richiesto. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $format) : ExportStrategyInterface` - metodo statico che istanzia e restituisce la strategy concreta corrispondente al formato richiesto

3.1.1.1.14 ExportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di esportazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'esportazione delle note. Segue la descrizione dei metodi:

- `export (string $content, string $title) : string` - trasforma il contenuto Markdown della nota nel formato di output specifico della strategy
- `contentType() : string` - restituisce il MIME type associato al formato di esportazione, per consentire al browser di interpretare correttamente il file ricevuto
- `extension() : string` - restituisce l'estensione del file associata al formato di esportazione, utilizzata per costruire il nome del file

3.1.1.1.15 MdExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato Markdown, è la strategy più semplice del sistema in quanto il contenuto delle note è già nativamente in formato Markdown, quindi non richiede alcuna trasformazione. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.16 HtmlExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato HTML. Utilizza la libreria *Parsedown* per convertire il contenuto Markdown della nota in markup HTML, che viene poi incorporato in un documento HTML completo e ben formato. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.17 PdfExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato PDF. Prima delega al parser la trasformazione del Markdown in HTML, poi utilizza la libreria *barryvdh/laravel-dompdf* per convertire l'HTML in un documento PDF binario. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.18 EditorContentFormatter Utility dedicata alla conversione di contenuti HTML generati da editor testuali in testo normalizzato in formato Markdown, ha l'obiettivo di eliminare elementi nascosti o metadati che utilizziamo nell'editor per la visualizzazione di blocchi in seguito alla generazione di testo con LLM. Segue la descrizione dei metodi:

- `htmlToMarkdown(string $html) : string` - riceve in input una stringa contenente HTML e restituisce una versione testuale normalizzata del contenuto
- `nodeToText(DOMNode $node) : string` - metodo ricorsivo utilizzato per convertire i nodi del DOM in contenuto testuale
- `normalizeMarkdownText(string $text) : string` - metodo di normalizzazione finale del testo estratto

3.1.1.2 Interazione con LLM^G

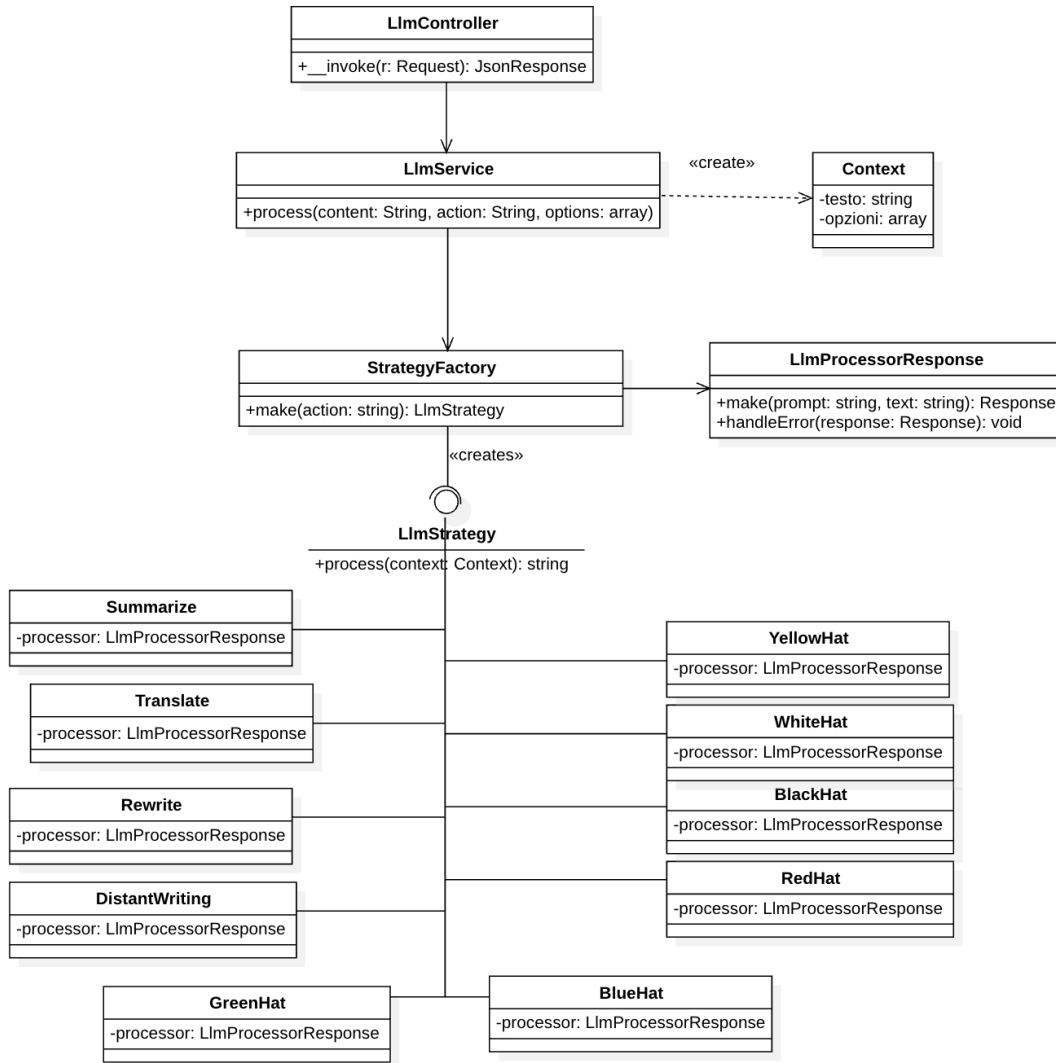


Figura 2: Diagramma delle classi per Interazione con LLM

3.1.1.2.1 LlmController È il controller HTTP responsabile^G della gestione delle richieste di elaborazione del testo tramite modelli linguistici. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta a una sola operazione. Si limita a validare i dati in ingresso, delegare l'elaborazione a *LlmService* e restituire il risultato in formato JSON. Segue la descrizione dei metodi:

- `__invoke(Request $request, string $id) : JsonResponse` - valida i parametri (il content, l'azione da svolgere e i parametri aggiuntivi dipendenti dall'azione), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200.

3.1.1.2.2 LlmService È il service responsabile^G della logica di elaborazione del testo tramite modelli linguistici. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra *LlmStrategyFactory* e le strategy di elaborazione, che ne eseguono il processo. Si limita



a costruire il contesto di esecuzione e delegare il lavoro alle classi di servizio. Segue la descrizione dei metodi:

- **process(string \$content, string \$action, array \$options = []) : string** - costruisce un oggetto *LlmContext* con il testo da elaborare e le opzioni aggiuntive, delega la selezione e creazione della strategy a *LlmStrategyFactory* in base all'azione e esegue la strategy. Infine restituisce il testo prodotto dalla strategy.

3.1.1.2.3 LlmStrategyFactory È la factory responsabile^G della creazione della strategy di elaborazione testuale, elaborata in base all'azione richiesta. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- **make(string \$action) : LlmStrategyInterface** - metodo statico che istanzia e restituisce la strategy concreta corrispondente all'azione richiesta.

3.1.1.2.4 LlmProcessorResponse Classe utility che gestisce le chiamate a un'API^G LLM^G tramite il client HTTP di Laravel. Segue la descrizione dei metodi:

- **make(string \$prompt, string \$text) : Response** - costruisce e invia una richiesta POST all'endpoint dell'API^G configurata, restituisce la Response grezza di Laravel
- **handleError(Response \$response) : void** - valida la risposta e lancia una eccezione in caso di errore, se la risposta è andata a buon fine non fa nulla

3.1.1.2.5 LlmStrategyInterface Interfaccia che definisce il contratto che ogni strategy di elaborazione testuale tramite modelli linguistici deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato alla elaborazione del testo. Segue la descrizione dei metodi:

- **process(LlmContext \$context) : string** - riceve un oggetto *LlmContext* contenente il testo da elaborare e le opzioni aggiuntive, esegue la chiamata al modello linguistico con il prompt specifico della strategy e restituisce il testo prodotto come string.

3.1.1.2.6 LlmContext È un oggetto di contesto che incapsula i dati necessari all'esecuzione di una strategy di elaborazione testuale; funge da contenitore di informazioni che vengono passata dalla *LlmService* alla strategy selezionata, evitando il passaggio di parametri multipli. Si limita a esporre i dati tramite getter.

3.1.1.2.7 Strategy concrete Tutte le strategy implementano *LlmStrategyInterface* e condividono la stessa struttura interna: costruiscono una chiamata all'API^G del modello linguistico con un prompt specifico per il tipo di richiesta di elaborazione tramite *LlmProcessorResponse* e restituiscono il contenuto generato dal modello.

- **Summarize** - elabora il testo producendo un riassunto che mantiene solo le informazioni più rilevanti, preservando la voce narrativa originale
- **Translate** - traduce il testo nella lingua specificata tramite il campo dell'array *options* del contesto
- **Rewrite** - riscrive il testo in base alle opzioni scelte dall'utente (variazione stilistica, estensione del testo, correzione grammaticale o miglioramento del lessico)
- **DistantWriting** - genera testo originale a partire da una descrizione fornita dall'utente
- **WhiteHat** - analizza il testo da un punto di vista fattuale, identificando dati oggettivi, informazioni verificabili e lacune conoscitive presenti nel contenuto
- **RedHat** - analizza il testo esprimendo le reazioni emotive e le impressioni intuitive che suscita



- **BlackHat** - individua rischi, criticità, punti deboli e potenziali conseguenze negative presenti nel testo
- **YellowHat** - identifica i punti di forza, benefici e aspetti positivi del testo
- **GreenHat** - propone idee alternative e riformulazioni possibili del contenuto
- **BlueHat** - valuta la chiarezza espositiva e il processo comunicativo del testo, fornendo una riflessione sull'organizzazione del contenuto

3.2 Architettura di deployment

La scelta della tipologia di **architettura di deployment** è stata guidata dal contesto di utilizzo del prodotto, dai requisiti di scalabilità previsti dal capitolato^G e dalle caratteristiche dello stack tecnologico. Non avendo ricevuto indicazioni specifiche riguardo a vincoli di deployment e non prevedendo significative espansioni o modifiche strutturali, il team ha optato per un'**architettura monolitica**. Questa soluzione, oltre ad essere in linea con le esigenze del prodotto, semplifica e velocizza la fase di progettazione^G e sviluppo^G ed evita la complessità introdotta da un'architettura a microservizi.

Il deployment viene gestito tramite **Docker Compose** perché, fornendo un ambiente di esecuzione preconfigurato e isolato, si semplificano notevolmente le operazioni di deployment e di manutenzione in qualsiasi ambiente, purché questo supporti Docker^G, e si evita la necessità di configurazione manuale.

3.2.1 Organizzazione dei container

Nonostante l'adozione di un'architettura monolitica dal punto di vista applicativo, il sistema è strutturato in più container^G al fine di separare le responsabilità tecniche e migliorare la gestione dell'ambiente di esecuzione.

- **app**
 - Contiene l'applicazione Laravel
 - Gestisce la business logic
 - Interagisce con il database PostgreSQL (opzionale)
- **web**
 - Utilizza nginx come web server
 - Espone l'applicazione sulla porta 8080
 - Funge da punto di ingresso per le richieste HTTP
 - Inoltra le richieste al container^G **app**
- **db**
 - Utilizza PostgreSQL
 - Responsabile^G della persistenza dei dati qualora si opti per questa soluzione invece che per la persistenza su filesystem
- **vite**(sviluppo^G)
 - Esegue un dev server per la compilazione dinamica di JavaScript e CSS
 - Espone le risorse sulla porta 5173

3.3 Architettura di dettaglio

4 Stato dei requisiti funzionali

Requisito	Descrizione	Rispettato
RF-O-1	Il sistema deve permettere all'utente di visualizzare l'archivio delle note presenti e per ogni nota il suo nome, la data di creazione e la data dell'ultima modifica	NO
RF-O-2	Il sistema deve permettere all'utente di aprire una nota	NO
RF-O-3	Il sistema deve permettere all'utente di uscire da una nota	NO
RF-O-4	Il sistema deve permettere all'utente di creare una nuova nota	NO
RF-O-5	Il sistema deve permettere all'utente di salvare la nota su cui sta lavorando	NO
RF-O-6	Il sistema deve permettere all'utente di salvare una nota non ancora presente nell'archivio per la prima volta	NO
RF-O-7	Il sistema deve permettere all'utente di clonare una nota	NO
RF-O-8	Alla clonazione di una nota, il sistema deve assegnare automaticamente un nome di default (es. "Nota data-ora") che l'utente può modificare	NO
RF-O-9	Il nome di default generato nella clonazione di una nota deve essere univoco all'interno dell'archivio e facilmente riconoscibile all'utente	NO
RF-O-10	Il sistema deve permettere all'utente di ricercare le note per nome nell'archivio	NO
RF-O-11	Il sistema deve permettere all'utente di eliminare una nota	NO
RF-O-12	Il sistema deve permettere all'utente di rinominare una nota	NO
RF-O-13	Il sistema deve permettere all'utente di esportare una nota aperta	NO
RF-O-14	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio	NO
RF-O-15	L'esportazione deve preservare integralmente il contenuto testuale della nota, senza perdita o alterazione di caratteri	NO
RF-O-16	L'esportazione deve preservare correttamente i caratteri Unicode del testo (es. accenti e simboli), senza sostituzioni o corruzione	NO
RF-O-17	Al termine dell'esportazione, il sistema deve notificare esplicitamente l'errore di esportazione all'utente	NO
RF-O-18	Il sistema deve permettere all'utente di importare una nota	NO
RF-O-19	L'importazione deve preservare integralmente il contenuto testuale della nota, senza perdita o alterazione di caratteri (incl. Unicode)	NO
RF-O-20	In caso di file non valido o corrotto, il sistema deve interrompere l'importazione senza creare note parziali e deve notificare chiaramente l'errore all'utente	NO

RF-O-21	Il sistema deve permettere all'utente di scrivere nell'editor con caratteri Unicode (es. lettere accentate e simboli)	NO
RF-O-22	Il sistema deve compilare la sintassi Markdown	NO
RF-O-23	Il sistema deve permettere la visualizzazione del documento formattato secondo la sintassi Markdown	NO
RF-O-24	L'aggiornamento della visualizzazione formattata deve avvenire in un tempo adeguatamente veloce	NO
RF-O-25	Il sistema deve permettere di cambiare modalità di visualizzazione	NO
RF-O-26	Il sistema deve fornire un'area di editing di testo in cui è possibile inserire e modificare del testo per la compilazione Markdown	NO
RF-O-27	Il sistema deve permettere l'applicazione di formattazioni Markdown senza scrivere manualmente la sintassi	NO
RF-O-28	L'editor deve rispondere all'input e alla formattazione dell'utente in modo fluido, senza ritardi percepibili durante la digitazione	NO
RF-O-29	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo grassetto senza scrivere manualmente la sintassi	NO
RF-O-30	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo corsivo senza scrivere manualmente la sintassi	NO
RF-O-31	Il sistema deve permettere l'applicazione e la rimozione della formattazione testo sottolineato senza scrivere manualmente la sintassi	NO
RF-O-32	Il sistema deve consentire l'applicazione e la rimozione della formattazione testo barrato senza richiedere l'inserimento manuale della sintassi	NO
RF-O-33	Il sistema deve consentire l'inserimento e la rimozione di link ipertestuali senza richiedere l'inserimento manuale della sintassi	NO
RF-O-34	L'editor deve rendere i link ipertestuali facilmente riconoscibili (es. evidenziandoli graficamente) e distinguibili dal testo normale	NO
RF-O-35	Il sistema deve consentire l'applicazione e la rimozione della formattazione testo commentato senza richiedere l'inserimento manuale della sintassi	NO
RF-O-36	Il sistema deve consentire l'inserimento di elenchi numerati	NO
RF-O-37	Il sistema deve consentire l'inserimento di elenchi puntati	NO
RF-O-38	Il sistema deve permettere all'utente di annullare l'ultima modifica fatta	NO
RF-O-39	Il sistema deve permettere all'utente di ripristinare la modifica precedentemente annullata	NO
RF-O-40	Il sistema deve permettere all'utente di selezionare porzioni di testo, sia tramite mouse che tastiera	NO
RF-O-41	Il sistema deve permettere all'utente di copiare il testo selezionato	NO
RF-O-42	Il sistema deve permettere all'utente di tagliare il testo selezionato	NO

RF-O-43	L'editor deve poter fare copiare e tagliare all'utente il testo in modo fluido, senza ritardi percepibili	NO
RF-O-44	L'editor deve preservare correttamente il contenuto e la struttura del testo copiato e tagliato, senza perdita o alterazioni di caratteri	NO
RF-O-45	Il sistema deve permettere all'utente di incollare del testo nell'area di testo	NO
RF-O-46	L'editor deve poter fare incollare all'utente il testo in modo fluido, senza ritardi percepibili	NO
RF-O-47	L'editor deve preservare e incollare correttamente il contenuto e la struttura del testo, senza perdita o alterazioni di caratteri	NO
RF-O-48	Il sistema deve permettere all'utente di esportare una nota aperta in MD	NO
RF-O-49	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in MD	NO
RF-O-50	Il sistema deve avvisare l'utente della presenza di una nota con lo stesso nome in seguito al primo salvataggio	NO
RF-O-51	Il sistema deve permettere all'utente di criticare il testo secondo il modello dei "Sei Cappelli"	NO
RF-O-52	Il sistema deve permettere all'utente di criticare il testo con una "Visione Emotiva" (Cappello Rosso)	NO
RF-O-53	Il sistema deve permettere all'utente di criticare il testo con una "Visione Neutra" (Cappello Bianco)	NO
RF-O-54	Il sistema deve permettere all'utente di criticare il testo con una "Visione Ottimista" (Cappello Giallo)	NO
RF-O-55	Il sistema deve permettere all'utente di criticare il testo con una "Visione dell'avvocato del Diavolo" (Cappello Nero)	NO
RF-O-56	Il sistema deve permettere all'utente di criticare il testo con una "Visione Originale" (Cappello Verde)	NO
RF-O-57	Il sistema deve permettere all'utente di criticare il testo con una "Visione Logica e Strutturata" (Cappello Blu)	NO
RF-O-58	Il sistema deve permettere all'utente di riassumere del testo tramite un agente esterno	NO
RF-O-59	Il sistema deve permettere all'utente di riscrivere del testo tramite un agente esterno	NO
RF-O-60	Il sistema deve permettere all'utente di scegliere il modo in cui verrà riscritto il testo (correzione grammaticale, estensione del testo, miglioramento del lessico, variazione stilistica)	NO
RF-O-61	Il sistema deve permettere all'utente di tradurre del testo tramite un agente esterno	NO
RF-O-62	Il sistema deve permettere l'invio da parte dell'utente di un prompt a un agente esterno per la generazione di testo (Distant Writing)	NO
RF-O-63	Il sistema deve permettere all'utente di sostituire nell'editor il testo originale con quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	NO

RF-O-64	Il sistema deve permettere all'utente di aggiungere nell'editor, in seguito alla selezione del testo originale, quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	NO
RF-O-65	Il sistema deve permettere all'utente di aggiungere nell'editor, in coda a tutto il testo presente nell'editor, quello generato dall'agente esterno (riassunto, riscrittura, traduzione)	NO
RF-O-66	Il sistema deve permettere all'utente di "rifiutare" il testo generato dall'agente esterno (riassunto, riscrittura, traduzione)	NO
RF-O-67	Il sistema deve permettere all'utente di copiare negli appunti il testo generato dall'agente esterno (riassunti, riscrittura, traduzione, generazione)	NO
RF-O-68	L'agente esterno deve produrre testo grammaticalmente, sintatticamente e semanticamente corretto	NO
RF-O-69	Il sistema deve avvisare l'utente in caso dell'impossibilità nell'eseguire una richiesta	NO
RF-O-70	Il sistema deve commentare il testo selezionato dall'utente utilizzato per interrogare la generazione (Distant Writing)	NO
RF-O-71	Se l'utente aggiunge una traduzione a una parte del testo selezionato, il sistema deve avvisarlo ogni volta che il testo tradotto viene modificato	NO
RF-O-72	Il sistema deve consentire all'utente di aggiornare una traduzione precedentemente inserita se il testo originale è presente ed è stato modificato	NO
RF-O-73	Il sistema deve consentire all'utente di rimuovere una traduzione precedentemente inserita	NO
RF-O-74	Il sistema deve consentire all'utente di rimuovere il testo originale di una traduzione precedentemente inserita	NO
RF-O-75	Il sistema deve notificare all'utente la perdita del testo originale di riferimento qualora venga eliminato, lasciando la traduzione priva di collegamento	NO
RF-O-76	Il sistema non deve consentire l'inserimento di una traduzione all'interno di una porzione di testo già tradotta	NO
RF-O-77	L'agente deve produrre traduzioni corrette nella lingua selezionata	NO
RF-O-78	Il sistema alla chiusura di una nota, se presenti delle modifiche non salvate deve avvisare l'utente della presenza di modifiche	NO
RF-O-79	Il sistema in caso di problemi di connessione con l'agente esterno deve avvisare l'utente	NO
RF-O-80	Durante una richiesta alla LLM, il sistema deve fornire all'utente un feedback chiaro sullo stato dell'operazione (es. indicatore di caricamento/elaborazione) fino al completamento o alla segnalazione di errore	NO
RF-O-81	Il feedback sullo stato della comunicazione con la LLM deve essere non bloccante e deve evitare che l'utente avvii richieste duplicate inconsapevolmente	NO



RF-O-82	Il sistema deve permettere all'utente di visualizzare l'editor con il contenuto della nota dopo l'apertura di questa	NO
RF-O-83	Il sistema deve avvisare l'utente in caso di tentativo di apertura di una nota non più esistente	NO
RF-O-84	Il sistema deve avvisare l'utente in caso di fallimento dell'apertura di una nota a causa di problemi di connessione al server	NO
RF-O-85	In caso di tentativo di chiusura di una nota con modifiche non salvate, il sistema deve avvisare l'utente e chiedere se desidera salvare le modifiche prima di chiudere	NO
RF-O-86	Il sistema deve permettere all'utente di salvare le modifiche effettuate all'interno di una nota	NO
RF-O-87	Il sistema deve avvisare l'utente in caso di presenza di una versione più recente della nota aperta nell'archivio	NO
RF-O-88	Il sistema deve permettere all'utente di sovrascrivere la nota presente nell'archivio con la versione aperta in caso di conflitto di versioni	NO
RF-O-89	Il sistema deve permettere all'utente di salvare la nota aperta con un nuovo nome in caso di conflitto con la versione presente nell'archivio	NO
RF-O-90	Il sistema deve permettere all'utente di aggiornare la nota aperta con le modifiche presenti nella versione più recente nell'archivio in caso di conflitto di versioni	NO
RF-O-91	Il sistema deve avvisare l'utente in caso di errori durante il salvataggio della nota causati da problemi di connessione al server	NO
RF-O-92	Il sistema deve avvisare l'utente in caso di tentativo di clonazione di una nota non più esistente	NO
RF-O-93	Il sistema deve avvisare in caso di impossibilità di clonare una nota a causa di problemi di connessione al server	NO
RF-O-94	Il sistema deve chiedere all'utente di confermare l'eliminazione di una nota	NO
RF-O-95	Il sistema deve avvisare l'utente in caso di impossibilità di eliminazione di una nota a causa di problemi di connessione al server	NO
RF-O-96	Il sistema deve impedire all'utente di rinominare una nota con un nome già presente nell'archivio e visualizzare un messaggio di errore	NO
RF-O-97	Il sistema deve avvisare l'utente in caso di impossibilità di rinominare una nota a causa di problemi di connessione al server	NO
RF-O-98	Il sistema deve avvisare l'utente in caso di impossibilità di esportare una nota a causa di problemi di connessione al server	NO
RF-O-99	Il sistema deve dare la possibilità all'utente di importare una nota in formato MD	NO
RF-O-100	Il sistema deve avvisare l'utente in caso di problemi di connessione al server durante l'importazione di una nota	NO

RF-O-101	Il sistema deve permettere all'utente di inserire elementi in un elenco puntato o numerato	NO
RF-O-102	Il sistema deve permettere all'utente di rimuovere elementi da un elenco puntato o numerato	NO
RF-O-103	Il sistema deve avvisare l'utente in caso di problemi di connessione con l'agente esterno durante la ritraduzione di una porzione di testo precedentemente tradotta	NO
RF-D-1	Il sistema deve aggiornare automaticamente i risultati della ricerca ad ogni modifica dell'input di ricerca	NO
RF-D-2	Il sistema deve permettere all'utente di esportare una nota aperta in PDF	NO
RF-D-3	Il sistema deve permettere all'utente di esportare una nota aperta in HTML	NO
RF-D-4	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in PDF	NO
RF-D-5	Il sistema deve permettere all'utente di esportare una nota direttamente dall'archivio in HTML	NO
RF-D-6	Il sistema deve permettere all'utente di utilizzare scorciatoie da tastiera per formattare il testo	NO
RF-D-7	Il sistema deve comunicare all'utente eventuali errori grammaticali nell'editor	NO
RF-D-8	Il sistema deve consentire all'utente di eseguire un minimo di 20 operazioni consecutive di annullamento e di ripristino delle modifiche sull'editor della nota corrente	NO
RF-D-9	In caso di incollaggio di contenuti non testuali, l'editor deve mantenere la reattività e mostrare una notifica non bloccante che spieghi l'azione effettuata (rifiuto o conversione in testo)	NO
RF-D-10	L'archivio deve aggiornarsi dinamicamente con i risultati della ricerca ad ogni modifica dell'input con una risposta percepita come immediata dall'utente	NO
RF-D-11	L'editor deve evidenziare in modo chiaro e distinguibile la porzione di testo utilizzato come prompt per l'interrogazione del LLM, così da renderla immediatamente riconoscibile all'utente	NO
RF-D-12	L'aggiornamento della visualizzazione formattata non deve bloccare l'input dell'utente durante la digitazione	NO
RF-OP-1	Il sistema deve aggiornare automaticamente la visualizzazione del testo formattato ad ogni modifica del Markdown, senza richiedere un'azione esplicita	NO
RF-OP-2	Il sistema deve comunicare all'utente eventuali errori di sintassi nei marcatori nell'editor	NO
RF-OP-3	L'editor deve permettere all'utente di interagire direttamente con la LLM tramite marcatori	NO
RF-OP-4	Il sistema deve permettere all'utente di registrarsi tramite nome utente e password	NO
RF-OP-5	Se password o nome utente non sono corrette il sistema non deve permettere la registrazione	NO
RF-OP-6	Il sistema deve permettere all'utente di accedere al proprio account	NO



RF-OP-7	Se password o nome utente non sono corrette il sistema non deve permettere l'autenticazione	NO
RF-OP-8	L'utente può effettuare il logout e terminare la sessione	NO
RF-OP-9	Il sistema deve permettere ad un utente autenticato di eliminare il proprio account	NO
RF-OP-10	Il sistema deve richiedere la password dell'utente per l'eliminazione del suo account	NO
RF-OP-11	Il sistema deve avvisare l'utente che la password inserita per l'eliminazione dell'account è sbagliata	NO
RF-OP-12	Il sistema deve richiedere la conferma all'utente per l'eliminazione del suo account	NO
RF-OP-13	Il sistema deve permettere all'utente autenticato di salvare note nel proprio account	NO
RF-OP-14	Il sistema deve permettere all'utente autenticato di aprire una delle note salvate nel proprio account	NO
RF-OP-15	Il sistema durante il salvataggio, nel caso in cui il DataBase non sia raggiungibile, deve visualizzare un messaggio d'errore	NO
RF-OP-16	Il sistema durante l'apertura di una nota, nel caso in cui il DataBase non sia raggiungibile, deve visualizzare un messaggio d'errore	NO
RF-OP-17	Le credenziali dell'utente devono essere trasmesse in modo sicuro durante l'autenticazione e la registrazione	NO
RF-OP-18	Il sistema deve completare una richiesta di autenticazione e registrazione in un breve lasso di tempo	NO
RF-OP-19	Il sistema non deve memorizzare né mostrare password in chiaro	NO
RF-OP-20	Il sistema deve limitare ad un massimo di 3 tentativi di accesso falliti per ridurre il rischio di attacchi a forza bruta	NO
RF-OP-21	In caso di credenziali errate, il messaggio di errore non deve rivelare se sia il nome utente o la password ad essere errata	NO
RF-OP-22	I messaggi di errore in autenticazione e registrazione devono essere chiari e indicare all'utente come correggere l'input	NO
RF-OP-23	I campi di inserimento credenziali devono fornire feedback immediato su validità del formato (es. vincoli su nome utente/password), senza attendere l'invio del form	NO
RF-OP-24	Dopo l'autenticazione, la sessione deve scadere automaticamente dopo un periodo di inattività pari a 1 mese	NO
RF-OP-25	Il sistema deve avvisare l'utente in caso di mancata registrazione/autenticazione dovuta a problemi di connessione al server	NO
RF-OP-26	Il sistema deve avvisare l'utente in caso di impossibilità di eliminazione dell'account a causa di problemi di connessione al server	NO

Tabella Requisiti Funzionali